



UNIDAD 4.- CADENAS DE CARACTERES

OBJETIVOS:

Que el alumno:

- Aprenda el uso de las cadenas de caracteres.
- Utilice adecuadamente las cadenas de caracteres en situaciones problemáticas y su solución en lenguaje C.

TEMAS:

1. Definición.
2. Lectura e impresión. Representación en lenguaje C.
3. Funciones predefinidas en el lenguaje C.



1. DEFINICIÓN DE CADENAS DE CARACTERES

En cualquier lenguaje de programación las cadenas de caracteres tienen una importancia especial, no solo porque es el tipo mediante el cual se almacenan los mensajes a pantalla o entradas del teclado, sino porque un caracter (char) es del tamaño de un byte. Una cadena de caracteres es una colección ordenada de bytes.

Hay lenguajes que tienen un tipo (type) preestablecido para tratar con cadenas de caracteres, es así en las distintas versiones de basic, donde el tipo 'String' es un tipo más (como 'integer'), En cambio en C/C++ podría decirse que el tipo 'String' no existe como tal (en C++ hay implementaciones de la clase 'String' pero no pertenecen propiamente al lenguaje sino a librerías anexas, dependiendo de cada compilador), el recurso usado por la mayoría de las funciones de estos lenguajes es representar una cadena de caracteres como un array de elementos tipo char (array de caracteres).

Una cadena de caracteres, representada en memoria, es una simple sucesión de bytes, cada caracter se corresponde con un byte, si queremos sacar en pantalla una cadena de caracteres, el problema es: ¿cómo sabe el programa donde finaliza esa cadena?, ¿cuál es su último byte? A esta pregunta diferentes lenguajes plantean diferentes respuestas, según un modelo de 'String' en C se reserva una función especial al caracter cuyo valor ASCII es 0 (cero), ese caracter indicará con su presencia que la cadena finaliza allí, ese char forma parte de la cadena, pero por convención no se lo tiene en cuenta al determinar el *largo* de la cadena. Ejemplo en la cadena "hola" se dice que tiene 4 caracteres, aunque en memoria luego de 'a' se encuentre el '\0' que forma parte de ella.

Una cadena en C es un conjunto de caracteres, o valores de tipo "char", terminados con el carácter nulo, es decir el valor numérico 0. Internamente se almacenan en posiciones

consecutivas de memoria. Este tipo de estructuras recibe un tratamiento especial, y es de gran utilidad y de uso continuo.

Array y cadena de caracteres: algunas diferencias

En muchos textos sobre C y C++ se encuentra la siguiente afirmación: *"En C/C++ una cadena de caracteres (string) es un array de caracteres"* es claro que esta afirmación es válida, una cadena de caracteres es un *array* y no una variable de tipo individual, como un entero o un float, sin embargo, hay ligeras diferencias entre los dos conceptos, el de array y el de cadena de caracteres.

Mientras que un **array de caracteres**: es un conjunto ordenado de 'n' bytes de cualquier valor, una **cadena de caracteres** es solo un subconjunto de ese array, desde el primer char hasta el primer '\0' encontrado en el array. En realidad, no es totalmente exacto decir 'subconjunto', pues si



por error nuestro array de caracteres no tiene un '\0', las funciones standard lo buscaran fuera del array, y así la cadena de caracteres llegara a tener mayor extensión que el propio array, desbordando su capacidad.

La manera de definir una cadena es la siguiente:

char <identificador> [<longitud máxima>;

Cuando se declara una cadena hay que tener en cuenta que tendremos que reservar una posición para almacenar el carácter nulo, de modo que, si queremos almacenar la cadena "FRT", tendremos que declarar la cadena como:

char cadena[4];

Tres caracteres para "FRT" y uno extra para el carácter '\0'. No es preciso poner el carácter '\0' (se añade por defecto)

También es posible hacer referencia a cada uno de los caracteres individuales que componen la cadena, simplemente indicando la posición. Por ejemplo, el tercer carácter de la cadena "FRT" será la 'T', se puede hacer referencia al mismo como cadena[2]. Los índices tomarán valores empezando en el cero, así el primer carácter de nuestra cadena sería cadena[0], que es la 'F'.

Una cadena puede almacenar informaciones como nombres de personas, mensajes de error, números de teléfono, etc.

La asignación directa **solo** está permitida cuando se hace junto con la declaración.

Por ejemplo:

char cadena[4] = "FRT";

2. LECTURA E IMPRESIÓN

Para mostrar una cadena en la pantalla se utiliza la función printf, utilizando %s en la cadena de formato

- Observe que no se pone el & delante de cad. Esto se debe a que cad es ya un puntero al primer elemento de la cadena
- printf mostrará todos los caracteres situados a partir de la dirección de inicio hasta que encuentre el carácter \0, el cual no imprime.

3. FUNCIONES PREDEFINIDAS EN EL LENGUAJE C

Existen unas cuantas funciones la biblioteca estándar de C para el manejo de cadenas:

- strlen
- strcpy
- strcat
- sprintf
- strcmp



Para usar las funciones hay que añadir la directiva:

```
#include <strings.h>
```

La función `strlen()` indica el largo de cadena y las funciones de impresión en pantalla funcionan normalmente, por la presencia del char `'\0'`.

strlen

Esta función nos devuelve el número de caracteres que tiene la cadena (sin contar el `'\0'`).

```
#include <stdio.h>
#include <strings.h>
```

```
main()
{
    char texto[]="Argentina";
    int longitud;

    longitud = strlen(texto);
    printf( "La cadena \"%s\" tiene %i caracteres.\n", texto, longitud );
}
```

strcpy

Copia el contenido de *cadena2* en *cadena1*. *cadena2* puede ser una variable o una cadena directa (por ejemplo "hola"). Debemos tener cuidado de que la cadena destino (*cadena1*) tenga espacio suficiente para albergar a la cadena origen (*cadena2*).

```
#include <stdio.h>
#include <string.h>
```

```
main()
{
    char origen[] = "Este es un curso de C";
    char destino[50];

    strcpy(destino, origen);
    printf( "Valor final: %s\n", destino);
}
```

La variable destino es una cadena constante ("Este es un curso de C") y no una variable. En este ejemplo la cadena destino es sustituida por la cadena origen.

Si la cadena origen es más larga que la destino, se eliminan las letras adicionales.



En el ejemplo siguiente la cadena destino recibe valor de una cadena.

```
#include <stdio.h>
#include <strings.h>

void main()
{
    char destino[50] = "Esto no es un curso de HTML sino un curso de C";

    printf( "%s\n", destino);
    strcpy( destino, "Este es un curso de C");
    printf( "%s\n", destino);
}
```

strcat

Esta instrucción sirve para concatenar una cadena a otra.

Con la instrucción `strcpy(nombre_completo, nombre );`

Asignamos a la variable *nombre_completo* el valor de la variable *nombre*.

La función `strcat` añade un espacio.

Finalmente se añade el apellido a la variable *nombre_completo*.

```
#include <string.h>
#include <stdio.h>

main()
{
    char nombre_completo[50];
    char nombre[]="Juan";
    char apellido[]="Perez";

    strcpy(nombre_completo, nombre);
    strcat(nombre_completo, " ");
    strcat(nombre_completo, apellido);
    printf( "El nombre completo es: %s.\n", nombre_completo );
}
```

sprintf

Funciona de manera similar a `printf`, pero en vez de mostrar el texto en la pantalla lo guarda en una variable (*destino*). El valor que devuelve (int) es el número de caracteres guardados en la variable *destino*.

Con `sprintf` podemos repetir el ejemplo de `strcat` de manera más sencilla:



```
sprintf(destino, format, cadena1, cadena2, .....);
```

```
#include <stdio.h>
#include <string.h>
```

```
main()
{
    char nombre_completo[50];
    char nombre[]="Juan";
    char apellido[]="Perez Gomez";

    sprintf(nombre_completo, "%s %s", nombre, apellido);
    printf( "El nombre completo es: %s.\n", nombre_completo);
}
```

strcmp

Compara *cadena1* y *cadena2*.

Si son iguales devuelve 0.

Un número negativo si *cadena1* va antes que *cadena2*; un número positivo si es al revés:

```
cadena1 == cadena2 > 0
cadena1 > cadena2 -> número negativo
cadena1 < cadena2 -> número positivo
```

```
int strcmp(cadena1, cadena2);
```

```
#include <stdio.h>
#include <strings.h>
```

```
main()
{
    char nombre1[]="Juan";
    char nombre2[]="Perez";

    printf( "%i", strcmp(nombre1,nombre2));
}
```